

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Architettura degli Elaboratori

Appunti a cura di
Lorenzo Ferraro

29 settembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Architettura degli Elaboratori*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@gmail.com**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Sistemi numerici	1
1.1	Introduzione ai sistemi numerici	1
1.1.1	Convertire i numeri tra i diversi sistemi numerici	1
1.1.2	Numeri col segno	2
1.2	Operazioni elementari	4
1.2.1	Addizioni e sottrazioni	4
1.2.2	Il problema dell'overflow	5
1.2.3	Moltiplicazioni	5
1.2.4	Shift di bit	6
1.3	Numeri con la virgola	6
1.3.1	Rappresentazione a virgola fissa	7
1.3.2	Numeri con la virgola con segno	7
1.3.3	Rappresentazione a virgola mobile	8
1.3.4	Arrotondare numeri a virgola mobile	10
1.3.5	Addizione con i numeri in virgola mobile	11
1.3.6	Moltiplicazione con numeri in virgola mobile	12

Capitolo 1

Sistemi numerici

1.1 Introduzione ai sistemi numerici

Il sistema numerico usato normalmente è il sistema numerico **decimale** (o in base 10). In questo, ogni cifra rappresenta una potenza di 10: $5374_{10} = 5 \cdot 10^3 + 3 \cdot 10^2 + 7 \cdot 10^1 + 4 \cdot 10^0$. Esistono, oltre a questo sistema, anche molti altri sistemi in basi diverse, tra cui il sistema **binario** e il sistema **esadecimale**.

Definizione 1.1 – Sistema binario

Il sistema **binario** rappresenta ogni numero tramite solo due cifre: **0** e **1**. Ogni cifra rappresenta una potenza di 2: $1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 1 = 13_{10}$.

Definizione 1.2 – Sistema esadecimale

Il sistema **esadecimale** rappresenta ogni numero tramite tutte le **10 cifre** e tramite le lettere che vanno dalla **A** alla **F** (che rappresentano i numeri da 10 a 15). Ogni cifra rappresenta una potenza di 16: $4AF_{16} = 15 \cdot 16^0 + 10 \cdot 16^1 + 4 \cdot 16^2 = 15 + 160 + 1024 = 1199_{10}$.

1.1.1 Convertire i numeri tra i diversi sistemi numerici

Abbiamo visto come convertire qualsiasi numero in binario/esadecimale in decimale, ma vediamo come fare il contrario.

- **Decimale** $\rightarrow$ **Binario**: vi sono due metodi:

- 1° Metodo: prendere la più grande potenza di 2 più piccola del numero da voler convertire, e sottrarla, e continuare così finché non si arriva a 0. Ad esempio, $68 = 64 - 4$, dunque $68_{10} = 1000100_2$.
- 2° Metodo: dividere il numero per 2 e posizionare il resto appena ottenuto a sinistra della rappresentazione binaria. Ad esempio, $68/2 = 34 + 0 \rightarrow 34/2 = 17 + 0 \rightarrow 17/2 = 8 + 1 \rightarrow 8/2 = 4 + 0 \rightarrow 4/2 = 2 + 0 \rightarrow 2/2 = 1 + 0 \rightarrow 1/2 = 0 + 1$, dunque $68_{10} = 1000100_2$.

- **Decimale** → **Esadecimale**: dividere il numero per 16 e posizionare il resto appena ottenuto a sinistra della rappresentazione esadecimale. Ad esempio, $68/16 = 4 + 4 \rightarrow 4/16 = 0 + 4$, dunque $68_{10} = 44_{16}$.
- **Esadecimale** → **Binario**: passare da esadecimale a binario è molto semplice. Bisogna prendere ogni cifra dell'esadecimale e convertirla singolarmente in binario su 4 bit. Ad esempio, $F3A_{16} = 111100111010_2$, dato che $F = 1111$, $3 = 0011$ e $A = 1010$. Questo metodo è valido per passare da una qualsiasi base potenza di 2 al binario: basta convertire ogni cifra in un numero di bit pari all'esponente di tale potenza. Con l'esadecimale ne prendiamo 4 ($16 = 2^4$), con la base 8 invece ne usiamo solo 3 ($8 = 2^3$).
- **Binario** → **Esadecimale**: anche passare da binario a esadecimale è molto semplice. Bisogna partire da destra e dividere il numero in gruppi da 4 bit. Si trasformano poi tutti i gruppi nella corrispondente cifra esadecimale e si riscrivono nello stesso ordine. Ad esempio, $100100_2 = 24_{16}$, dato che $0100_2 = 4_{16}$ e $10_2 = 2_{16}$. Questo metodo è valido per passare dal binario a una qualsiasi base potenza di 2: basta dividere il numero in gruppi di lunghezza pari all'esponente di tale potenza. Per l'esadecimale i gruppi sono lunghi 4 ($16 = 2^4$), per la base 32 sono invece lunghi 5 ($32 = 2^5$).

Proprietà 1.1 – Potenze di 2

Conoscendo solo alcune potenze di 2, è possibile approssimare i valori di molte più potenze. Infatti, se si conoscono tutte le potenze da 2^0 a 2^9 , e sapendo che $2^{10} \approx 1000$, $2^{20} \approx 1000000$, $2^{30} \approx 1000000000$, ..., si possono approssimare tutte le potenze tra 2^0 e 2^{39} . Infatti, $2^{34} = 2^{30} \cdot 2^4 \approx 1000000000 \cdot 16 = 16000000000$.

Definizione 1.3 – Definizioni sistema binario

Nel sistema binario, chiamiamo **bit** le cifre dei numeri. Un **nibble**, poi, sono 4 bit mentre un **byte** sono 8 bit. Esistono poi il **most significant bit** e **least significant bit**, che rispettivamente il bit più a sinistra e più a destra, e il **most significant byte** e **least significant byte**, che sono rispettivamente il byte più a sinistra e più a destra.

Ad esempio, nel numero 1010101010, il primo 1 è il MSB, mentre l'ultimo 0 è il LSB. Allo stesso modo, nel numero CEB48E8 (rappresentato in esadecimale, dove ogni cifra rappresenta un nibble), il byte CE è il MSB, mentre il byte E8 è il LSB.

1.1.2 Numeri col segno

In qualsiasi sistema numerico, per rappresentare un numero negativo basta mettere il simbolo $-$ prima del numero. Questo, però, per i computer non è possibile: lavorando solo con dei bit, non si può anteporre ad un numero il simbolo $-$. Infatti, il computer non ha idea di cosa sia

quel simbolo, conosce solo 0 e 1. Dunque bisogna trovare un'altra soluzione per rappresentare i numeri col segno. Vi sono due modi possibili:

- **Segno e Ampiezza**
- **Complemento a 2**

Proprietà 1.2 – Segno e Ampiezza

Segno e Ampiezza è una tecnica usata dai computer per rappresentare i numeri col segno. Si usa il **bit più significativo per rappresentare il segno**: se questo vale **0**, allora il numero è **positivo**, altrimenti è **negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = (-1)^{a_{N-1}} \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 6 tramite il Segno e Ampiezza diventa in binario 0110, mentre -6 diventa 1110. È dunque necessario utilizzare un bit in più del normale per rappresentare i numeri. Con N bit si possono quindi rappresentare tutti i numeri da $-(2^{N-1} - 1)$ a $2^{N-1} - 1$.

Il problema più grande di questa tecnica è l'addizione: semplicemente, questa non funziona. Se infatti proviamo a sommare -6 (1110) e 6 (0110), troviamo che

$$\begin{array}{rcccccc} 1 & 1 & 1 & 0 & + & & \\ 0 & 1 & 1 & 0 & = & & \\ \hline 1 & 0 & 1 & 0 & 0 & = & 19_{10} \neq 0 \end{array}$$

Inoltre, un'altro problema è che vi sono due rappresentazioni dello 0: infatti, $0 = 1000 = 0000$.

Per questi motivi nasce un'altra tecnica, quella del **complemento a 2**.

Proprietà 1.3 – Complemento a 2

Complemento a 2 è un'altra tecnica usata per rappresentare i numeri col segno. Il **bit più significativo** ha lo stesso valore visto con i numeri senza segno, ma con il **segno negativo**. La formula per convertire un numero in decimale è la seguente:

$$A = a_{N-1}(-2^{N-1}) + \sum_{i=0}^{N-2} a_i 2^i$$

Ad esempio, il numero 7 tramite complemento a 2 diventa in binario 0111, mentre -7 diventa 1001. Con N bit si possono quindi rappresentare tutti i numeri da -2^{N-1} a $2^{N-1} - 1$.

Con questa implementazione si risolvono entrambi i problemi del Segno e Ampiezza. Infatti, lo zero è rappresentato solo dalla sequenza 0000, e l'addizione non ha problemi: infatti, se proviamo a sommare -6 (1010) e 6 (0110), troviamo che

$$\begin{array}{r} 1 \ 0 \ 1 \ 0 \ + \\ 0 \ 1 \ 1 \ 0 \ = \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \ = \ 0 \end{array}$$

dove l'uno iniziale non si conta dato che stiamo parlando di numeri a 4 bit.

Proprietà 1.4 – Convertire un numero nell'opposto tramite il Complemento a 2

Preso un numero in binario possiamo facilmente trasformarlo nell'opposto tramite il Complemento a 2. Infatti, basta **invertire i bit** e **aggiungere 1**. Preso ad esempio $6_{10} = 0110_2$, troviamo che $-6_{10} = 1001_2 + 1 = 1010 = -6_{10}$.

Questa proprietà funziona per un motivo molto semplice: sappiamo che $6 + \bar{6} = 0110 + 1001 = 1111$, dunque aggiungendo un 1 troviamo che $6 + \bar{6} + 1 = 1111 + 1 = 10000$, e dato che il primo zero si elude, troviamo che $6 + \bar{6} + 1 = 0$, ossia $-6 = \bar{6} + 1 \rightarrow -1001 = 0110 + 1$.

1.2 Operazioni elementari

1.2.1 Addizioni e sottrazioni

Le addizioni in binario e in esadecimale funzionano esattamente uguale a come funzionano in decimale. Ad esempio,

$$\begin{array}{r} 1 \\ 0 \ 1 \ 1 \ + \\ 0 \ 1 \ 0 \ = \\ \hline 1 \ 0 \ 1 \end{array}$$

$$\begin{array}{r} 1 \\ A \ 4 \ F \ + \\ 2 \ A \ 1 \ = \\ \hline C \ F \ 0 \end{array}$$

Anche per la sottrazione il metodo rimane lo stesso. Ad esempio,

$$\begin{array}{r} 1(2) \\ 0 \ 10 \ 0 \ - \\ 0 \ 0 \ 1 \ = \\ \hline 0 \ 0 \ 1 \end{array}$$

$$\begin{array}{r} 1(16) \\ 3 \ D \ 9 \ - \\ 1 \ A \ D \ = \\ \hline 2 \ 2 \ C \end{array}$$

1.2.2 Il problema dell'overflow

Osservazione 1.1 – Overflow

Un problema classico nelle addizioni tra numeri binari in un sistema digitale è l'**overflow**: i computer lavorano su un numero fisso di bit, e con un addizione è possibile sfiorare questo numero.

Ad esempio, se lavoriamo con numeri a 4 bit, la somma $1001 + 1000 = 10001$ sfiora, creando diversi problemi. Infatti, con N bit abbiamo un range di numeri rappresentabili, da 0 a $2^N - 1$ (questo se parliamo di numeri senza segno).

Vi sono due modi per risolvere questo problema:

- **Sign-Extension**
- **Zero-Extension**

Proprietà 1.5 – Sign-Extension

La **sign-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero (in Complemento a 2) così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **duplicare il bit più significativo a sinistra raddoppiando il numero di bit del numero** (raddoppiamo il numero di bit così da esser coperti sia sulle addizioni ma anche sulle moltiplicazioni, che vedremo tra poco).

Ad esempio, $3_{10} = 0011 = 00000011$, e $-5_{10} = 1011 = 1111011$. Questo con gli 0 ovviamente funziona, ma anche con gli 1: infatti, $-5 = 1011 = -8+2+1 = -16+8+2+1 = -32+16+8+2+1 = -64+32+16+8+1 = 1111011$.

Proprietà 1.6 – Zero-Extension

La **zero-extension** è un'operazione usata dai computer per aumentare il numero di bit di un numero così da poter completare somme su di esso che superino il limite di bit prestabilito. La regola è semplice: **raddoppiare la lunghezza del numero aggiungendo tutti 0 a sinistra**.

Ad esempio, $3_{10} = 0011 = 00000011$ (come nella Sign-Extension). La Zero Extension però **non funziona con i numeri col segno**. Infatti, $-5_{10} = 1011 \neq 00001011 = 11_{10}$.

1.2.3 Moltiplicazioni

Anche le moltiplicazioni in binario e in esadecimale funzionano come in decimale. Ad esempio,

$$\begin{array}{r}
 0 \ 1 \ 1 \ \times \\
 0 \ 0 \ 1 \ = \\
 \hline
 0 \ 1 \ 1 \ + \\
 0 \ 0 \ 0 \ + \\
 0 \ 0 \ 0 \ + \\
 \hline
 0 \ 0 \ 0 \ 1 \ 1
 \end{array}$$

$$\begin{array}{r}
 3 \ B \ 7 \ - \\
 2 \ C \ 4 \ = \\
 \hline
 E \ D \ C \ + \\
 2 \ C \ 9 \ + \\
 7 \ 6 \ E \ + \\
 \hline
 A \ 4 \ 6 \ 1 \ C
 \end{array}$$

1.2.4 Shift di bit

Definizione 1.4 – Shift

Uno **shift** è un'operazione sui bit che **sposta tutte le posizioni dei bit** di una sequenza binaria o a destra o a sinistra di un determinato numero di posizioni.

Esistono due tipi di shift:

- **shift aritmetico**, dove si ripete il bit più significativo seguendo la logica della sign-extension;
- **shift logico**, dove si inseriscono degli 0 a sinistra della sequenza seguendo la logica della zero-extension.

Ad esempio, $-7_{10} = 1001 \gg 2 = 0010 = 2_{10}$ è uno **shift logico** (si è infatti perso il segno, cosa che capita quando si usa la zero-extension), mentre $-8_{10} = 111000 \ggg 3 = 111111 = -1_{10}$ è uno **shift aritmetico**. Notiamo che, in entrambi i casi, il nuovo numero si può ottenere prendendo l'originale e dividendolo per 2^N , dove N è il valore dello shift. Questa è una proprietà degli shift di destra. Allo stesso modo, per gli shift di sinistra, basta moltiplicare il valore per 2^N . Ad esempio, $-8_{10} = 111000 \ll 2 = 100000 = -32 = -8 \cdot 2^2$.

Proposizione 1.1 – Calcolare numero decimale dopo uno shift

Preso un numero A_{10} , effettuando uno shift a **sinistra** di N posizioni, troviamo che $A_{10} \ll N = A_{10} \cdot 2^N$. Se invece si effettua uno shift a **destra**, troviamo che $A \gg N = A/2^N$.

1.3 Numeri con la virgola

Siamo riusciti a rappresentare numeri interi senza segno e con il segno, ma dobbiamo ora rappresentare i **numeri con la virgola**. Abbiamo due possibili rappresentazioni:

- **Virgola fissa**
- **Virgola mobile**

1.3.1 Rappresentazione a virgola fissa

Proprietà 1.7 – Rappresentazione a virgola fissa

La rappresentazione a **virgola fissa** è una tecnica usata dai computer per rappresentare i numeri con la virgola. Si divide il numero in due sezioni: una rappresenta la parte intera, mentre l'altra rappresenta la **parte decimale**. Il numero di bit per la parte intera e quello per la parte decimale deve essere deciso a priori. Ad ogni cifra della parte frazionaria corrisponde una **potenza di due negativa**, partendo da 2^{-1} .

Esempio

Rappresentare **6.75** utilizzando **4 bit interi** e **4 bit frazionari** (ossia per la parte decimale).

Sappiamo già che $6_{10} = 0110_2$. Per la parte decimale, abbiamo 4 cifre che ci permettono di calcolare tutti i valori da 0 a 1 con salti di $1/16$. 0.75 diventa quindi 1100, dato che $1100 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = 0.5 + 0.25 = 0.75$. Dunque troviamo che **6.75 = 01101100**.

In generale, per convertire da decimale a binario un numero con la virgola si possono usare entrambi i metodi già visti in precedenza, con una differenza per il 2° metodo: infatti, non dovremo più dividere per 2, ma dovremo moltiplicare la parte decimale per 2, e se questo numero è minore di uno allora segniamo 0, altrimenti segniamo 1 e sottraiamo 1 al valore ottenuto. Nell'esempio di prima, $0.75 \cdot 2 = 1.5 \geq 1$, dunque segniamo 1 e sottraiamo 1 ottenendo 0.5. Ora, dato che $0.5 \cdot 2 = 1$, segniamo 1 e sottraiamo 1, ottenendo 0, dunque tutte le cifre rimanenti sono 0, e otteniamo quindi il valore 1100.

1.3.2 Numeri con la virgola con segno

Dobbiamo ora combinare la rappresentazione a virgola fissa con il segno e ampiezza e con il complemento a 2.

- Con il **segno e ampiezza** è facilissimo: semplicemente il primo bit della parte intera sarà riservato al segno.
- Con il **complemento a 2** bisogna utilizzare l'algoritmo visto nella Proprietà 1.4.

Ad esempio, il numero -7.5 diventa con il segno e ampiezza **11111000**, mentre con il complemento a 2 bisogna prima calcolare 7.5, che è 0111000, poi invertire i bit e sommare 1, trovando quindi come valore **10001000**. Questo valore ha senso dato che $1 \cdot -2^3 + 1 \cdot 2^{-1} = -8 + 0.5 = -7.5$.

Proposizione 1.2

Utilizzare 4 bit per la parte decimale di un numero significa togliergli 4 bit per la parte intera. È quindi come se lo stesso **dividendo per $2^4 = 16$** . Dunque un qualsiasi numero a 8 bit con la rappresentazione a virgola fissa è pari a quello stesso numero calcolato con 8 bit di parte intera diviso per 16.

Riprendendo l'esempio di -7.5 , 10001000 con 8 bit di parte intera diventa -120 , e infatti $-7.5 = -120/16$.

1.3.3 Rappresentazione a virgola mobile

Per studiare la rappresentazione a virgola mobile è necessario conoscere la **notazione scientifica**.

Definizione 1.5 – Notazione scientifica

La **notazione scientifica** è una tecnica usata per rappresentare i numeri nella forma $\pm M \times B^E$, dove M è la **mantissa**, B è la base e E è l'esponente.

N.B. In un sistema a base n , $1 \leq M < n$. Dunque ad esempio nel sistema decimale la mantissa è compresa tra 1 e 10.

Ad esempio, nel nostro sistema decimale scriveremmo $2437 = 2.437 \times 10^3$.

Proprietà 1.8 – Rappresentazione a virgola mobile

La rappresentazione a **virgola mobile** è una tecnica usata dai computer per rappresentare i numeri con la virgola che sfrutta la **notazione scientifica**. Si stabilisce un numero di bit a priori per rappresentare un numero, come ad esempio **32 bit**, e si dividono questi bit per rappresentare il **segno** (che richiede un bit), l'**esponente** (che, in una rappresentazione a 32 bit, richiede 8 bit) e la **mantissa** (che si prende il resto dei bit, ossia 23 in una rappresentazione a 32 bit). La base, trovandoci in un sistema binario, è ovviamente **2**.

Per usare questa rappresentazione bisogna:

- 1° **Convertire il numero da decimale a binario**
- 2° **Scriverlo nella notazione scientifica binaria** (vedremo ora come farlo)
- 3° **Scriverlo nella notazione segno-esponente-mantissa**

Vi sono però altre due importanti considerazioni da fare:

- l'1 iniziale della mantissa è **sottointeso**. Questo perché è sempre presente in tutti i numeri dato che $M \geq 1$.
- l'esponente è **polarizzato**, ossia non si usa l'esponente base ma gli si aggiunge un certo **bias**, che nel caso dei numeri a 32 bit vale 127. Questo lo si fa per poter rappresentare anche gli esponenti negativi.

Esempio

Rappresentare il numero -228_{10} usando la **rappresentazione a virgola mobile con 32 bit**.

Per prima cosa, quindi, convertiamo 228 in binario (il segno non ci interessa, vi è un bit apposta in questa rappresentazione che se ne occupa): $228_{10} = 11100100_2$. A questo punto dobbiamo scriverlo nella notazione scientifica binaria, in un processo che prende il nome di **normalizzazione**: $11100100_2 = 1.11001_2 \times 2^7$. Aggiungiamo il bias all'esponente che diventa $127 + 7 = 134$, e riscriviamo tale numero nella sua rappresentazione a 32 bit finale:

$$1|10000110|11001000000000000000000_2$$

il cui valore esadecimale è $0xC3640000$ (dove $0x$ indica l'inizio del valore esadecimale).

Vi sono diverse convenzioni da stabilire per questa rappresentazione, dunque si utilizza uno standard che prende il nome di **IEEE 754**.

Definizione 1.6 – IEEE 754

L'**IEEE 754** è lo standard utilizzato per la rappresentazione a virgola mobile.

Racchiudiamo in questa tabella tutti i casi speciali della rappresentazione a virgola mobile, che sono stati così standardizzati sotto l'IEEE 754.

Number	Sign	Exponent	Fraction
0	Indifferente	00000000	000000000000000000000000
∞	0	11111111	000000000000000000000000
$-\infty$	1	11111111	000000000000000000000000
NaN	Indifferente	11111111	non-zero
Denormalizzati	Indifferente	00000000	$M \neq 0$ $(-1)^s \times 2^{-126} \times 0.m$

Dunque lo 0 si rappresenta con il codice esadecimale $0x00000000$ o il codice $0x80000000$ (vi sono anche qui, come nel Segno e Ampiezza, due rappresentazioni per esso).

Quando invece vogliamo rappresentare un'operazione indefinita, come $\infty - \infty$ o $0/0$.

Nella tabella sono anche rappresentati un altro genere di numeri: i numeri **denormalizzati** (o subnormali). Questi numeri sono numeri la cui mantissa non inizia con 1, perché voglio

rappresentare numeri ancora più piccoli di 10^{-126} (che sarebbe il minimo consentito dai numeri normalizzati dato che l'esponente 00000000 è riservato allo 0). Infatti, si è pensato che dividendo il valore della mantissa per 2 si potesse diminuire ancora di più. Ad esempio, 0.1×10^{-126} ha sempre come esponente -126 , ma ha come mantissa 0.1 e non 1. In questo caso, però, si deve quindi togliere l'1 sottinteso di cui avevamo parlato precedentemente, e si perde anche precisione: infatti i bit rimanenti per la mantissa vera e propria diminuiscono sempre di più a favore di numeri più piccoli. In questo modo, però, è possibile arrivare fino a 2^{-149} con soli 32 bit.

Proprietà 1.9 – Livello di precisione

In base al numero di bit, si può raggiungere un livello di precisione più o meno alto. Vi sono 3 standard conosciuti:

- **Half-Precision** (16 bit)
- **Single-Precision** (32 bit, ciò che utilizzeremo noi)
- **Double-Precision** (64 bit)

1.3.4 Arrotondare numeri a virgola mobile

Abbiamo già affrontato il problema dell'overflow, ossia quando un numero sfiora il numero di bit consentiti (esiste poi anche l'**underflow**, ossia quando un numero è così piccolo da non poterlo rappresentare). Questo problema è risolvibile in parte grazie alla sign-extension o alla zero-extension, ma ci sono numeri che da un sistema binario non sono rappresentabili. Ad esempio, $1/3$ non è rappresentabile: si avrebbe bisogno di un numero di bit infinito. È per questo che in informatica spesso bisogna **arrotondare**.

Proprietà 1.10 – Arrotondare

Esistono quattro modi per arrotondare un numero:

- 1) per **difetto**
- 2) per **eccesso**
- 3) più **vicino allo zero** (ossia se è positivo per difetto, se è negativo per eccesso)
- 4) al **più vicino possibile** (default, e in caso di parità arrotonda al valore il cui bit meno significativo è pari, ossia 0)

Per capire meglio questi concetti, vediamo come arrotondare 1.100101 (1.578125) a solo 3 bit dopo la virgola con tutti e 4 i modi.

- 1) **1.100**
- 2) **1.101**

3) **1.100**

4) **1.101** (dato che la parte scartata 101 è maggiore della metà dell'intervallo 100)

1.3.5 Addizione con i numeri in virgola mobile

L'addizione tra numeri in virgola mobile segue dei passaggi molto specifici:

- 1° Si estraggono l'esponente e i bit frazionari
- 2° Si comparano gli esponenti, e si shifta la mantissa più piccola se necessario
- 3° Si sommano le mantisse
- 4° Si normalizza la mantissa e si aggiusta l'esponente se necessario
- 5° Si arrotonda il risultato
- 6° Si riscrive il numero nel formato a virgola mobile

Può sembrare difficile, ma vediamo un esempio per chiarirci le idee. Sommiamo i numeri $0x3FC00000$ e $0x40500000$.

Per prima cosa dobbiamo quindi estrarre l'esponente e i bit frazionari di entrambi i numeri:

$$3FC00000_{16} = 0|01111111|100000000000000000000000$$

$$40500000_{16} = 0|10000000|101000000000000000000000$$

La mantissa del primo numero è quindi 1.1, mentre per il secondo è 1.101. Gli esponenti sono rispettivamente 127 e 128, ossia 0 e 1 tolto il bias, e dato che $0 - 1 = -1$, dobbiamo fare uno shift verso destra della mantissa del primo numero di 1. Dunque $1.1 \gg 1 = 0.11 (\times 2^1)$. Possiamo ora sommare le due mantisse:

$$\begin{array}{r} 0. \ 1 \ 1 \ + \\ 1 \ 1 \ 0 \ 1 \ = \\ \hline 1 \ 0. \ 0 \ 1 \ 1 \end{array}$$

Dobbiamo ora normalizzare la mantissa: $10.011 \times 2^1 = 1.0011 \times 2^2$. Non serve arrotondare nulla, dato che i bit frazionari sono ben meno di 23, dunque possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000001|001100000000000000000000 = 0x4098000_{16}$$

Se invece prendiamo due numeri con segno opposto, come ad esempio $0x40C00000$ e $0xC0200000$.

Per prima cosa estraiamo l'esponente e i bit frazionari:

$$\begin{aligned} 40C00000_{16} &= 0|10000001|100000000000000000000000 \\ C0200000_{16} &= 1|10000000|010000000000000000000000 \end{aligned}$$

La mantissa del primo numero è 1.1, mentre quella del secondo è 1.01. Gli esponenti sono rispettivamente 129 e 128, ossia 2 e 1, e dato che $2 - 1 = 1$, dobbiamo fare uno shift verso destra della mantissa del secondo numero di 1. Dunque $1.01 \gg 1 = 0.101 (\times 2^2)$. Possiamo ora sottrarre (dato che la seconda è negativa) le due mantisse:

$$\begin{array}{r} 1. \ 0 \ 0 \ 0 \ - \\ 0. \ 1 \ 0 \ 1 \ = \\ \hline 0. \ 1 \ 1 \ 1 \end{array}$$

Dobbiamo ora normalizzare la mantissa: $0.111 \times 2^2 = 1.11 \times 2^1$. Non dobbiamo arrotondare nulla, quindi possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000000|110000000000000000000000 = 0x40600000$$

1.3.6 Moltiplicazione con numeri in virgola mobile

Anche la moltiplicazione tra numeri in virgola mobile è abbastanza schematica:

- 1° Si estraggono gli esponenti e i bit frazionari
- 2° Si sommano gli esponenti (togliendo poi un bias, dato che sennò ne avremmo due)
- 3° Si fa il prodotto tra le mantisse
- 4° Si calcola il segno (se i segni sono discordi allora è -, altrimenti è +)
- 5° Si normalizza il tutto se necessario
- 6° Si riscrive il numero nel formato a virgola mobile

Proviamo a moltiplicare $0x4040000$ e $0x4020000$.

Per prima cosa estraiamo quindi l'esponente e i bit frazionari da entrambi i numeri:

$$\begin{aligned} 40400000_{16} &= 0|10000000|100000000000000000000000 \\ 40200000_{16} &= 0|10000000|010000000000000000000000 \end{aligned}$$

La mantissa del primo numero è 1.1, mentre quella del secondo è 1.01. Gli esponenti sono entrambi 128, dunque l'esponente finale è $128 + 128 - 127 = 129$. Eseguiamo ora il prodotto tra

le mantisse:

$$\begin{array}{r}
 1. \ 1 \ 0 \ \times \\
 1. \ 0 \ 1 \ = \\
 \hline
 1 \ 1 \ 0 \ + \\
 0 \ 0 \ 0 \\
 1 \ 1 \ 0 \ = \\
 \hline
 1. \ 1 \ 1 \ 1 \ 0
 \end{array}$$

Non dovendo normalizzare nulla, possiamo riscrivere tutto nel formato a virgola mobile:

$$0|10000001|111000000000000000000000 = 0x40F00000$$